

TinyTydi walkthrough

Branch `feature/tydi-generator-integration` of `tydi-tutorial`. It adds TinyTydi as a submodule in `tinytydi/`: the Tydi formalism as a runnable simulator, with a JSON front end and a Chisel interface that is checked against it cycle for cycle.

These steps go from a fresh clone to the message text of a JSON document showing up as characters on a typed waveform. Budget half an hour, most of which is the first Scala build. Use the VNC container flavour if you want to see the waveform.

1. Clone and open the container

```
git clone --recurse-submodules -b feature/tydi-generator-integration \
  git@github.com:abs-tudelft/tydi-tutorial.git
cd tydi-tutorial
```

--recurse-submodules matters, `tinytydi/` is a submodule. If you forget it, run `git submodule update --init tinytydi`.

Open the folder in VS Code, Reopen in Container, pick **Tydi-Tools (VNC desktop)**. The desktop is at <http://localhost:6080/vnc.html>.

2. Check the container came up

`postCreateCommand` runs `.devcontainer/bootstrap.sh`, which checks out the submodule and exports two stimulus bundles. The bundles are generated, not committed, and the Chisel tests fail without them.

```
ls tinytydi/hdl/bundles/      # chatmsgs, demo
bash .devcontainer/bootstrap.sh # re-runnable, if either is missing
```

3. The simulator by hand

Pure Python, nothing compiles here. This is the part a tutorial audience sees first.

```
cd tinytydi
./main.py                                # six subcommands
./main.py normalise --type "Dim(Group(Bits(64), Dim(Dim(Bits(8))))))"
```

`normalise` prints one rewrite per line and ends at `Stream(Group(Bits(64), Stream(Bits(8), d=2)), d=1)`.

```
./main.py example
```

The chat message from the paper, one step per Enter. Four columns: inputs, parse tree, buffers, output history. Characters show as letters, terminated nodes turn red. Ctrl-C when you have seen enough, it is a few dozen steps. The dashboard assumes a 110-column terminal.

```
./main.py simulate
```

Guided run. Enter accepts each default, ? at any prompt explains the option. Worth trying ? on the lanes and the property-set prompt.

```
./main.py fsm --type "Dim(Dim(Bits(8)))" --lanes 4
```

The derived control-loop FSM as a Markdown table: 2 states, 3 transitions.

4. A JSON document as a Tydi type

```
./main.py json ../tydi-material/chat-messages/chat-messages.json --type-only
```

Three physical streams. Stream 1 is 64 bits wide and packs three document fields (`timestamp`, `message_id`, `user_id`) that were not adjacent in the document; stream 2 is the message text, 8 bits at dimension 4.

```
./main.py json examples/chat.json --lanes 1,4 --interactive
```

The same dashboard as `example`, but driven by a document. Use this small file rather than the tutorial's chat data, which is 513 characters.

```
./main.py json ../tydi-material/student-example/student.json --type-only
```

This one has to fail, with exit code 2. `"study_end": null` is an optional, which is a Union, and Union is outside the implemented fragment. The error names the path.

5. Export the bundle

```
./main.py export --json ../tydi-material/chat-messages/chat-messages.json \
--lanes 1,2,4 --out hdl/bundles/chatmsgs
```

514 cycles, 182 transfers. Four files, and they are the whole Python-to-Chisel interface:

File	What it is
tables.json	the parameters the circuit is elaborated from
input_*.hex	the elements, one per line per stream
golden.trace	what the semantics did with them, cycle by cycle
fields.json	which document field occupies which bits

6. Run the Chisel tests

```
cd hdl
scala-cli test .
```

First run downloads about 700 MB of Scala dependencies into the `tinytydi-coursier` volume; after that it is about 20 seconds. Six tests. The two that matter print `[chatmsgs] 514 cycles matched the golden trace`, and the same for `demo`.

The same `TinyTydiInterface.scala` is elaborated for both bundles, nothing is generated. Every directory under `bundles/` is a test case, so exporting a new type is enough to test it.

The version pins in `project.scala` are not arbitrary, the container ships a forked Chisel. You can ignore the hints about outdated dependencies and the `Unsupported statement ... during PDG building` warnings from `ChiselTrace`.

7. The waveform, and the characters

```
./invoke-surfer chatmsgs --chars
```

The first line should read `Tywaves: on` (typed `hgldd: ...`). Anything else and you get a waveform without types; the line says which of the two reasons applies. `--chars` preloads the signals carrying the message text, and it finds them from `fields.json`, so it works for any document.

Then, in Surfer:

1. Expand `outStream_2`.
2. Right-click a data ... value row, **Format > ASCII**. This cannot be set from the command line.
3. Same on `in_2 > data` for the element side.

Signal	What it shows
<code>outStream_2 > data(i).value</code>	the character in lane <code>i</code>
<code>outStream_2 > strb</code>	which lanes carry data in this transfer
<code>outStream_2 > endi</code>	index of the last active lane
<code>outStream_2 > last(i)</code>	dimensions closed by lane <code>i</code> , one bit per dimension
<code>in_2</code>	the element side, one character per cycle
<code>ptr_2</code>	lanes staged so far, 0 to 3
<code>_state_output, fullMap</code>	the parse FSM, one bit per node of the normalised type

8. Things that look wrong but are not

- **valid rises in the same cycle the last lane fills.** The transfer lane is driven from a wire, not from `bufData_2_3`, so it already carries the element parsed this cycle. Watching the buffer register makes it look a cycle early.
- **valid high for two cycles in a row is two transfers.** "Shall" is five characters into four lanes: `s h a l`, then `l` alone with `last` closing the word. `strb` and `endi` tell them apart.

- **Lanes with `strb = 0` still show a letter.** Stale bytes from the previous transfer.
- **`out_*_ready` sits at 0 and the design never reads it.** The interface is an open-loop source, because the formalism has no handshake. Fine here, not fine against a real sink.
- **Read time from the timestamps, not the clock.** Two timestamps per cycle, `golden.trace` cycle N is at timestamp $2N+1$. `reset` does not pulse in the dump either.
- **`surfer-tywaves` rewrites the VCD.** Given `--hgldd-dir` it writes `./TinyTydiInterface.vcd` and displays that, not the file you passed. Opening that file by hand later gives you whichever bundle wrote it last, untyped, which is why you want the script.

If something goes wrong

Symptom	Cause
no bundles found	bootstrap never ran, see step 2
detected dubious ownership	<code>git config --global --add safe.directory '*'</code>
no arguments allowed for nullary constructor <code>ChiselStage</code>	you ran <code>sample-circuits/tydi-example.test.scala</code> ; known-broken upstream and unrelated to this branch
a rebuild re-downloads 700 MB	the coursier volume is missing from the devcontainer <code>.json</code> you launched
Tywaves: off	that run emitted no HGLDD; the waveform still opens, without types

Longer version, including which signals exist and why a trace of an eight-node type has 285 of them: [tinytydi/hdl/README.md](#).